

Getter and Setter in Java

Last Updated : 22 Jun, 2023



In Java, Getter and Setter are methods used to protect your data and make your code more secure. Getter and Setter make the programmer convenient in setting and getting the value for a particular data type.

Getter in Java: Getter returns the value (accessors), it returns the value of data type int, String, double, float, etc. For the program's convenience, the getter starts with the word "get" followed by the variable name.

Setter in Java: While Setter sets or updates the value (mutators). It sets the value for any variable used in a class's programs. and starts with the word "set" followed by the variable name.

Syntax

```
class ABC{
    private variable;

    public void setVariable(int x){
        this.variable=x;
    }

    public int getVariable{
        return variable;
    }
}
```

Note: In both getter and setter, the first letter of the variable should be capital.

Examples of Getter and Setter in Java

Example 1:

```
// Java Program to Illustrate Getter and Setter

// Importing input output classes
import java.io.*;

// Class 1
// Helper class
class GetSet {

    // Member variable of this class
    private String name;

    // Method 1 - Getter
    public String getName() { return name; }

    // Method 2 - Setter
    public void setName(String N)
    {

        // This keyword refers to current instance itself
        this.name = N;
    }
}

// Class 2
// Main class
class GFG {

    // Main driver method
    public static void main(String[] args)
    {

        // Creating an object of class 1 in main() method
        GetSet obj = new GetSet();

        // Setting the name by calling setter method
        obj.setName("Geeks for Geeks");
        // Getting the name by calling getter method
        System.out.println(obj.getName());
    }
}
```

Output

Geeks for Geeks

Getter and Setter give you the convenience of entering the value of the variables of any data type by the requirement of the code. Getters and setters let you manage how crucial variables in your code are accessed and altered. It can be seen in the program discussed below as follows:

Example 2

```
// Java Program to Illustrate Getter and Setter

// Importing input output classes
import java.io.*;

class GetSet {
    // Member variable of this class
    private int num;

    // Method 1 - Setter
    public void setNumber(int number)
    {
        // Checking if number is between 1 to 10 exclusive
        if (number < 1 || number > 10) {

            throw new IllegalArgumentException();
        }
        num = number;
    }

    // Method 2 - Getter
    public int getNumber() { return num; }
}

// Class 2
// Main class
class GFG {
    // Main driver method
    public static void main(String[] args)
    {
        GetSet obj = new GetSet();
    }
}
```

```
// Calling method 1 inside main() method
obj.setNumber(5);

// Printing the number as setter above
System.out.println(obj.getNumber());
    }
}
```

Output

5

Explanation of the above program:

Here we can see that if we take a value greater than 10 then it shows an error, By using the `setNumber()` method, one can be sure the value of a number is always between 1 and 10. This is much better than updating the number variable directly.

Note: This could be avoided by making the number a private variable and utilizing the `setNumber` method. Using a getter method, on the other hand, is the sole way to read a number's value.